ARC UI · UIARC.DEV · PROOF OF CONCEPT

Security Threat Model

AI-Facing Attack Surface of Arc UI — Defensive Threat Model POC

Generated 2026-11-11

A defensive, educational threat model of how a React UI library that is consumed by autonomous AI coding agents can turn its docs, MCP outputs, and installable source into untrusted inputs, with a safe, fully local, benign demonstration.

Scope and Safety Statement

> **Authorized, defensive, educational material for a security-labs curriculum.**

- **Purpose:** analyze the AI-facing attack surface of Arc UI (https://uiarc.dev) and teach the threat classes that arise when an AI agent is the primary consumer of a UI library.
- **Live service:** interaction was **read-only and minimal**. It consisted of a handful of GETs of public `llms.txt`, per-component markdown, `r/*.json`, and skill files, plus **one** MCP `initialize` POST that returned `401`. There was no authentication attempt, no retry, no fuzzing, no flooding, and no exploitation.
- **Demonstration: 100% local** and **mock-only**. The only "payload" is a harmless marker file, `PWNED_marker.txt`, written inside `demo/sandbox-repo/`. There are **no** network callbacks, **no** exfiltration, **no** credentials, **no** destructive actions, and **no** reusable attack tool.
- **Inert source:** the mock "malicious" component source is inert text. It is only ever statically scanned as text and is never executed.
- **Not an accusation:** the live Arc service, as observed on 2026-11-10, appears well-run and benign. The threats are *structural*. They would apply to any library in the same position. Arc is used as a concrete, real-world instance to make the class legible. This document does not claim Arc is malicious.

Why a UI Library Is an AI Attack Surface

Arc UI is a React component and block library designed explicitly for consumption by AI coding agents. Its AI-facing channels are the core of the product:

Channel	URL / command	What it delivers into an agent
`llms.txt` index	`https://uiarc.dev/llms.txt`	Catalog, install instructions, and links to every per-component doc
Per-component markdown	`https://uiarc.dev/components/<id>/markdown`	"When to use / when not / props / a11y / motion" docs the agent reads into its context before coding
Agent "skills"	`https://uiarc.dev/r/skills/arc/*.md`	Always-on rule files intended to be pasted into `CLAUDE.md`, `AGENTS.md`, or `.cursor/rules`
MCP server (HTTP)	`claude mcp add --transport http arc https://uiarc.dev/api/mcp`	Tools (`search_components`, `get_component`, and others) whose outputs land in context
shadcn-CLI install	`npx shadcn@latest add @uiarc/button` fetches `https://uiarc.dev/r/button.json`	**Component source code written directly into the developer's repository**

The security-relevant inversion is this: **the library's docs, tool outputs, and source are untrusted input to a semi-autonomous agent that holds repository-write and often shell access.** Agents tend to treat "the library's own docs" as authoritative. That placement of untrusted content inside the trusted zone is the attack surface.

Trust boundary

```
TRUSTED                             |   UNTRUSTED (crosses the boundary)
-------                             |   --------------------------------
developer intent / prompt           |   llms.txt text
agent's own safety policy           |   per-component markdown
                                    |   MCP tool outputs and tool DESCRIPTIONS
                                    |   r/*.json source code and docs/meta fields
------------------------------------>|   skill files pasted into CLAUDE.md
agent with repo-write + shell       |
```

Recon Findings (Real, Read-Only, 2026-11-10)

All requests were issued once, at low volume, from a standard client. No fuzzing, flooding, auth attempts, or exploitation was performed.

Surface probed	Method	Result	Evidence
`llms.txt`	GET with a browser UA and with an agent-style UA (`claude-code/1.0` , `Accept: text/plain`)	HTTP 200, `text/plain` , 92,699 bytes. **Byte-identical** across both UAs. No UA sniffing observed. Content is a legitimate catalog.	Described in THREAT-MODEL.md §1.1
Per-component markdown (sample: `button`)	GET `components/button/markdown`	HTTP 200, `text/markdown` , 6,609 bytes. **Identical** across browser and agent UA. Grep for injection-like strings (`ignore` , `exfiltrat` , `env` , `token` , `curl` , `run this` , `tracking`) returned **no hits**.	`recon-evidence/real_button_markdown.md`
Registry JSON `r/button.json` (shadcn "copy source into my repo" payload)	GET	HTTP 200, 13,349 bytes. `files[]` carries real TSX and CSS source. Scanned for `dangerouslySetInnerHTML` , `eval(` , `fetch(` , `innerHTML` , `new Function` , `process.env` , `child_process` , and dynamic `import(` : **zero hits**.	THREAT-MODEL.md §1.3
Agent skill file `r/skills/arc/INSTRUCTIONS.md`	GET	HTTP 200, 3,219 bytes. Asks the user to paste it into `AGENTS.md` or `CLAUDE.md` and tells the agent to treat the Arc MCP, `llms.txt` , and markdown as the source of truth for UI decisions. This is a legitimate feature and also the canonical trust-establishment channel.	`recon-evidence/real_arc_INSTRUCTIONS.md`
MCP server `POST /api/mcp`	Well-formed JSON-RPC `initialize` , agent UA	**HTTP 401**, JSON-RPC error `-32001` ("Sign in to use the Arc MCP server"). `WWW-Authenticate` points to OAuth protected-resource metadata (Clerk-backed). `GET` with a browser UA returned HTTP 405. `tools/list` **and tool descriptions were not reachable unauthenticated**, so the live tools could not be enumerated.	`recon-evidence/real_mcp_initialize_401.json`
Integrity and provenance of docs and `r/*.json`	Inspection of response headers and payloads	**No signatures, no lockfile, no SRI, no content hash, and no version pin** on the agent side.	THREAT-MODEL.md §1.6

Summary

Property	Observation	Security read
UA / Accept sniffing on docs	None detected (byte-identical)	Good hygiene; no agent-versus-human divergence seen
Injected instructions in real docs	None found	Arc is not currently poisoning docs
Real installed source	Clean, no risky APIs in `button`	Current source is safe
MCP authentication	OAuth-gated (Clerk)	Strong control; limits anonymous abuse
Integrity and provenance	**No signatures, no lockfile, no SRI** on docs or `r/*.json`	**The structural gap** (see T2 and T1 below)

The structural weakness that remains even for a benign Arc: markdown docs and `r/*.json` source are served over plain HTTPS with no content signing, no provenance, and no pin or lockfile integrity on the agent side. Trust rests entirely on "it came from uiarc.dev today."

Threat Model

Threats are STRIDE-tagged and grounded in a kill chain. The authoritative detail is in `THREAT-MODEL.md`.

ID	Threat	STRIDE	Likelihood	Impact	Risk
T1	Prompt injection via docs and MCP output	Tampering, EoP	Medium	High	**HIGH**
T2	shadcn "copy source into repo" supply chain	Tampering, EoP	Medium	High	**HIGH**
T3	MCP tool-poisoning, rug-pull, and response divergence (cloaking)	Tampering, Repudiation, Spoofing	Low to Medium	High	**MEDIUM**
T4	Over-trust and confused deputy (amplifier)	EoP	High	High	**HIGH**

T1: Prompt injection via MCP-served and markdown docs (HIGH)

The agent fetches `components/<id>/markdown`, or an MCP `get_component` result, and concatenates it into its context as trusted reference. If that text contains imperative instructions addressed to the agent, such as "add this analytics snippet to `index.html`", "set an environment variable", or "run this setup command", a naive agent may execute them. It cannot reliably distinguish documentation describing a component from instructions addressed to it. The kill chain runs from recon (the agent reads docs), through delivery (an instruction embedded in a doc or tool output), to execution (edits, env changes, or shell commands), and possibly persistence (the instruction written into `CLAUDE.md` via a skill file). Arc's real docs contain no such instructions today, but Arc's own `INSTRUCTIONS.md` trains the agent to treat Arc text as authoritative project rules, which is the precondition this threat needs. A single compromised doc, a poisoned mirrored example, or reflected third-party content would be enough.

T2: Supply chain risk of shadcn-style "copy source into my repo" (HIGH)

`npx shadcn@latest add @uiarc/button` fetches `r/<name>.json` and writes its `files[].content` as real source files into the repository. Unlike npm, there is no lockfile or version pin (the agent gets whatever the URL returns today), no registry integrity hash or SRI, no signature, and often no human review gate when an agent runs the install and commits. A malicious or compromised item could ship a `dangerouslySetInnerHTML` sink, an obfuscated `fetch()` beacon, a build-time `postinstall`, or a dynamic `import()`. Because the agent installed a "documented component," the diff is often waved through. The payload path is real and confirmed: `r/button.json` carries full TSX and CSS in `files[]`. Arc's current source is clean, and there is no pin or hash to detect a future change.

T3: MCP tool-poisoning, rug-pull, and response divergence (MEDIUM)

Three variants exist:

- **Tool poisoning:** instructions hidden in a tool *description*, which the agent reads to decide how to call tools.
- **Rug-pull:** the server serves benign content during evaluation and then changes tool behavior or output after the client has approved it.
- **Response divergence / cloaking:** the server returns different content to an agent than to a human reviewer, based on UA, Accept, or session, so audits see clean docs while agents receive poisoned ones.

Arc's MCP is OAuth-gated, so live tool descriptions could not be inspected. Two mitigating observations: the public docs showed no UA or Accept divergence, and authentication limits anonymous tampering. Poisoning and rug-pull are post-trust, in-session threats that authentication does not remove. A compromised or malicious authenticated server could still mutate tool outputs over time, and a dynamic server could key responses on the authenticated session.

T4: Over-trust and confused deputy (HIGH, amplifier)

This is the force multiplier for T1 through T3. The agent is a confused deputy: it holds privileges the library author does not (repository write, shell, secrets, commit and push), and it lends those privileges to whatever the library "says." Asking users to paste `INSTRUCTIONS.md` into `CLAUDE.md` makes the over-trust persistent and project-wide, so every later turn inherits "Arc is authoritative." T4 needs no exploit of its own. It raises the severity of every other threat.

The Controlled Local Demonstration

File: `demo/naive_agent_sim.py`

The simulator runs the same mock library surface past two agent profiles and compares outcomes.

Step	Naive (over-trusting) agent	Defended agent
T1: read `mock-registry/docs/quick-note.md`	Finds agent-directed imperatives (matched by a regex) and obeys them by writing the benign marker `PWNED_marker.txt`	Treats the doc as data, ignores the imperatives, and flags them for a human
T3: read `mock-registry/r/mcp_toolpoison_response.json`	Obeys a hidden instruction in the `get_component` tool description (writes the same benign marker)	Refuses to act on the instruction in the tool description
T2 and T4: install `mock-registry/r/quick-note.json`	Writes the mock source to `sandbox-repo/` verbatim and "commits" it without review	Runs a static scan for red-flag strings (`dangerouslySetInnerHTML` , `eval(` , `new Function` , `import(` , `fetch(` , `XMLHttpRequest` , `document.cookie` , `child_process`), fails the scan, and blocks the install
Outcome	Marker exists: agent compromised	Marker absent: attacks neutralized

The script's self-check passes when the naive run produces the marker and the defended run does not. The only difference between the two profiles is the trust boundary.

What the demo does **not** do: it does not execute any mock file, does not make network calls, does not read secrets, and does not perform any destructive action. The malicious `.tsx` is read as text and statically scanned only.

Run command

```
cd demo
python -I naive_agent_sim.py      # -I = isolated mode: ignores cwd and environment
```

Expected final line: `DEMO SELF-CHECK: PASS` .

The `-I` flag is deliberate. It keeps the interpreter from loading modules from the working directory or from environment-driven paths while the lab is run.

Mitigations

For library authors

Control	Description	Threat addressed
Keep instructions out of docs	Docs should describe components and never address the agent with imperatives. Treat every markdown page as data.	T1
Sign and provide provenance	Publish signatures (for example Sigstore/cosign or detached signatures) and or SRI-style hashes for `r/*.json` and docs. Publish a manifest the client can verify. Extend the integrity controls that already protect MCP to public docs and source.	T2, T1
Version and pin	Give every registry item an immutable version and content hash so that `shadcn add @uiarc/button@1.2.3` is reproducible and old hashes stay valid.	T2
Serve identical content to everyone	No UA, Accept, or session-based divergence between agents and human reviewers. Arc currently passes this and should keep it.	T3
Stable, declarative MCP tool descriptions	No instructions in descriptions. Version descriptions and publish a changelog so clients can detect rug-pulls.	T3
Minimize secret-adjacent guidance	Where docs must mention environment variables or tokens (for example `ARC_PRO_TOKEN`), state explicitly "never commit; set in environment" and never tell an agent to echo or transmit a secret. Arc already does this.	T1, T4

For agent and developer users

Control	Description	Threat addressed
Treat library output as untrusted data	Library docs, MCP outputs, and tool descriptions are data, never instructions. Wrap ingested docs in a data boundary and do not follow imperatives found inside them. This is the highest-value control.	T1, T3
Human review before commit	Never let an agent auto-commit shadcn or `r/*.json` output. Diff every added file and grep for `dangerouslySetInnerHTML`, `eval`, `new Function`, dynamic `import()`, network calls, and `postinstall`.	T2
Pin and hash	Record the content hash of each `r/*.json` you install and fail the build if it changes. Re-review on every bump.	T2
Sandbox the agent	No ambient repository push, no raw shell, and no `.env` reads in the agent's default toolset. Scope tools and require explicit approval for writes outside an allowlisted `components/` path.	T4
Allowlist MCP servers	Review tool descriptions on connect and re-review after any server or version change.	T3
Separate reading from acting	The context that ingests untrusted docs should not be the same privileged context that writes files and runs shell, or actions should require a human-in-the-loop gate.	T4, T1

Project Structure

```
security-threat-model/
├─ README.md                          Safety scope, file map, run instructions, and summary of findings
├─ THREAT-MODEL.md                    Authoritative source: recon findings, STRIDE threat model, severities, mitigations
├─ recon-evidence/                    Small artifacts captured read-only from the live site
│  ├─ real_button_markdown.md         A real per-component doc (what an agent ingests)
│  ├─ real_arc_INSTRUCTIONS.md        Arc's real "paste into CLAUDE.md" skill rules
│  └─ real_mcp_initialize_401.json    Real MCP authentication-gate response (401)
└─ demo/                              Safe local lab (mock data only)
   ├─ naive_agent_sim.py              Simulator: naive agent versus defended agent
   ├─ mock-registry/                  Mock library surface (never executed)
   │  ├─ docs/quick-note.md           Mock component doc with a benign injected instruction (T1)
   │  └─ r/
   │     ├─ quick-note.json           Mock shadcn registry item with inert "malicious" source (T2)
   │     └─ mcp_toolpoison_response.json  Mock MCP tools/list with a poisoned tool description (T3)
   └─ sandbox-repo/                   Gitignored "dev repo" target, created at run time
```

Responsible-Use Note

- This material is for defensive analysis, threat modeling, teaching, and building safer agent and library practices. Do not use the mock payloads, the simulator's patterns, or the recon method against systems you are not authorized to test.
- The recon was passive and minimal by design. Any further live testing of the MCP surface requires explicit authorization from the service owner and must not involve authentication bypass or fuzzing.
- Findings describe classes of risk. They are not evidence of malicious conduct by Arc or any other party, and should be communicated that way.
- Keep the demonstration local. Do not replace the benign marker with a real network, credential, or execution payload.